ARCHITECTURE REVIEW · SPARK V0.9.2 · 2026-10-17

Spark 源码精读 架构评估

全量走查 147 个 Python 文件、29,600 行核心代码与 6,600 行前端实现， 输出缺陷清单、架构缺口与竞品差距评估，并给出定位策略、 历史治理与生态建设建议。

badhope · 五平台发布矩阵

GitCode · Gitee · GitHub ×3 — 全部公开 · MIT

一、开源状态与仓库治理

五个平台（GitCode/badhope、Gitee/badhope、GitHub/badhope、GitHub/X33834、GitHub/Morningstar202604）的 spark 仓库已全部核验为公开状态，许可证统一为 MIT（仓库内 LICENSE 文件，版权声明 badhope）。Gitee 仓库原为私有，本次已通过 API 转公开；其余四平台本就公开。五平台 HEAD 一致于 a0f5769，属于完全同步的镜像，后续以工作区的 spark_push.sh 一键推送即可保持同步。

治理建议：GitHub badhope 为主仓库（Issues/Releases/Actions 均在此运行），其余四平台仅作镜像分发；给三台 GitHub 各配一个 branch protection，避免多账号误推互相覆盖。

二、项目全景：目录树与模块职责

Spark Agent v0.9.2 是本地自托管 AI 编程 Agent（对标 Claude Code / Codex CLI），Python 3.11+，四形态交付（Web UI / 桌面 EXE / 终端 TUI / CLI），另有 mcp-serve 把自己暴露为 MCP 服务器。核心规模：src 147 个 Python 文件 29,594 行，前端 React+Vite 约 6,600 行，测试 68 个文件 11,046 行。

spark/ ├── src/spark/ │ ├── core/ 引擎：loop.py(1115行,ReAct主循环) context(上下文组装) │ │ compact(LlamaIndex压缩) checkpoints(tar快照) graph(LangGraph备选) │ ├── tools/ 33个内置工具： fs/shell/search/gitops/bg/web/notebook │ │ mcp_bridge(MCP客户端) registry(模式注册+分发) │ ├── providers/ litellm统一流式(openai_compat) factory 路由矩阵(routing) │ ├── memory/ 向量+关键词双路记忆： store(Chroma+SQLite) extractor service │ ├── edit/ aider SEARCH/REPLACE 编辑引擎(三级容错+语法门) │ ├── web/ 手写ThreadingHTTPServer+线程SSE+PTY终端，React前端dist │ ├── tui/ textual 终端界面（审批弹窗/计划卡/内嵌终端） │ ├── prompts/ system.md 系统提示词（仅34行） │ ├── config/policy/sandbox/store 配置清洗/审批策略/沙箱/SQLite会话库 │ └── evals/git_memory/skills/agents/hooks/mcp_registry … 评测/任务级git历史/技能/子代理/钩子 ├── web/src/ 26个React组件（DiffView/PlanCard/ExecutionGraph/虚拟列表…） ├── tests/ 68个测试文件（含对抗性安全、MCP边界、性能边界） ├── docs/ 架构/配置/API/竞品对比/ADR 决策记录 └── .github/workflows/ ci/security/evals/release/build-exe 五条流水线

代码质量总体印象：工程素养高于同体量开源项目平均线——配置清洗、tar 路径逃逸校验、签名级审批绑定、FTS 转义、对抗性测试等都做到了位；主要短板集中在模型侧协议假设、上下文策略和分发链，而非代码风格。

三、精读发现的问题与 Bug（按优先级）

级	位置	问题	影响与修复建议
P0	tools/shell.py	bwrap 命令用 --unshare-all 却在注释声明不隔离网络	装了 bwrap 的 Linux 上 workspace 模式所有联网命令（pip/npm/git clone）直接失败；加 --share-net
P0	store.py	删消息时 FTS 索引按全局 rowid>X 删除，无会话过滤	单库多会话时把其他会话的搜索索引删掉；DELETE 加 session 条件或按 messages 子查询
P0	memory/service.py	embedding 默认请求发往 DeepSeek chat 端点（无 /embeddings）	默认配置下向量检索永远失败降级关键词；为 embedding 单独配 base_url 或默认关向量路
P1	core/tokens.py + loop	全部用量为启发式估算（4字符/token），无真实 usage	成本计量不可信；流式响应里已有 usage 字段应回填，估算仅兜底

级	位置	问题	影响与修复建议
P1	core/compact.py	压缩摘要绕过 Provider 抽象直连 litellm	mock provider 测试会意外打真接口；压缩应走 provider.completion
P1	providers/openai_compat	多工具调用缺 id 时统一兜底 call_0；参数 JSON 解析失败静默置空	同轮多调用撞 id 破坏审批签名与结果配对；用序号兜底并回显解析错误
P1	core/loop.py	压缩只在 turn 开始触发，超长轮次中途不再压缩	长轮次靠 trim_recent 硬裁剪丢工具结果导致模型重做；在每轮工具结果后检查阈值
P1	prompts/system.md	系统提示词仅 34 行，无代码风格/提交/安全细则	直接封顶行为质量上限；参照 Claude Code 扩充至 200+ 行并做版本化评测
P2	tools/registry.py	execute 每次调用重建全部分发表	每次工具调用白耗几百微秒；构建一次缓存
P2	config.py	history_budget_chars 按字符、max_context_tokens 按 token，两套单位	统一为 token 口径，删掉字符预算
P2	sandbox.py	shell 防泄漏为黑名单子串匹配（自述 best-effort）	python -c 等可绕过读环境变量；至少对子进程做 env 白名单注入
P2	core/context.py	默认 max_context_tokens=32768 偏小	现代模型 128k 起步，过小导致频繁压缩；默认提到 131072

表 1 精读缺陷清单（P0 建议本周内修复，均为小改动）

四、架构师视角：缺失环节与竞品差距

对照仓库自带的 docs/competitive-comparison.md（Qoder/ZCode/opencode/Codex CLI）与本次源码走查，差距集中在四类：分发链、上下文工程、真实用量与反馈闭环、IDE 生态。模型能力本身不是瓶颈——BYO 模型下，同模型同提示词的差距主要来自下面这些工程项。

缺口	现状	竞品参照	优先级
分发与安装	无 PyPI/安装脚本/自动更新，仅源码与 CI 产物	pip install opencode / codex 一条命令	最高
真实用量计量	全 estimated，无成本视图	codex /status、ZCode 用量面板	高
消息回溯/分支	存储层已支持，UI 未接	codex thread/revert、ZCode Undo	高
IDE 集成	无（仅自带四形态）	opencode ACP、Codex VS Code 扩展	中
@文件引用/外部目录	无	三家均有 @ 补全	中
小模型分工	单模型干所有事	压缩/标题/抽取用小模型，主循环用强模型	中
LSP/诊断闭环	无	Claude Code/Qoder 读编译器诊断	中
子代理轨迹回放	只回传 summary	Claude Code 子代理全轨迹可查	低

表 2 功能缺口矩阵（相对四家竞品）

架构层面还有两个结构性问题：其一，LangGraph 版循环（graph.py）与生产 while 循环长期并存，两套真相源迟早漂移，建议要么接入要么删除；其二，Web 层是手写 ThreadingHTTPServer + 线程

SSE，可靠但难扩展，多会话并发时无隔离，若后续要做远程/多人使用应尽早换 ASGI 框架。

五、现有能力与功能全览

交互形态：Web UI（SSE 流式、Diff 视图、计划卡、执行图、上下文计量、命令面板、双语 i18n）、Textual TUI、桌面 EXE（PyInstaller）、CLI（exec 管道模式、resume、doctor、init 引导、config 命令组）、mcp-serve 对外暴露。

Agent 内核：ReAct 主循环 + token 熔断与重复调用指纹去重、85% 阈值自动压缩（litellm 结构化摘要，LlamaIndex 兜底）、tar 增量检查点与恢复、声明式子代理（.claude/agents 兼容，单发与最多 4 路并行）、6 档审批模式 × 工具分类策略 × 通配权限规则、4 档沙箱（含 bwrap 命名空间与 Windows ACL）。

工具集 33 个：文件（读/写/补丁/树）、grep/glob、shell（后台任务+输出截断）、git 六件套 + worktree + run_tests、notebook 读写、web 搜索抓取（SSRF 白名单）、MCP 桥接（命名空间工具、国内可达性探测、注册表搜索）、任务子代理。

记忆与上下文：向量（Chroma）+关键词（FTS）双路记忆、自动事实抽取与整合、git-memory 任务级 worktree/step-commit/squash、AGENTS.md/CLAUDE.md 三级加载、tree-sitter 仓库地图、技能注入、钩子事件。

工程质量：五条 CI 流水线、bandit/安全扫描、68 个测试文件 1.1 万行、对抗性安全测试、评测框架（MockProvider 回放 + 6 类断言 judge）。

六、定位与获客：最实在的打法

定位一句话：不跟 Claude Code 拼模型，拼三件它给不了的事——密钥与代码不出本机、国产模型零门槛直连（路由矩阵+离线档）、以及 MCP/Skills 生态兼容带来的迁移零成本。目标人群按优先级：① 对数据出境敏感的国内企业开发团队（内网部署+Ollama 离线是硬需求，竞品全部吃不到）；② 已被订阅费劝退的个人开发者（BYO DeepSeek/Kimi，月成本几块钱）；③ 想要本地化 Claude Code 体验的重度 Agent 用户。

获客只做三件事，都要能衡量：第一，把 pip install spark-agent 做通并写进 README 第一行，没有一键安装就没有增长；第二，录 3 条真实任务演示（修 bug/写功能/审查 PR），展示 Diff、检查点回滚、任务级 git 历史这三个可视化差异点——用户不为架构买单，为看得见的回滚买单；第三，用 mcp-serve 打进已有 MCP 工具链社区，让 Spark 成为别的 Agent 的执行后端，这是零成本分发渠道。

要避免的坑：不要在 README 强调 33 个工具、6 档审批这类参数，用户只关心能不能装、贵不贵、和我现在的工作流兼不兼容；也不要宣传尚不稳定的向量记忆（默认配置下本就降级）。

七、历史与文件管理的治理建议

现状确实混乱：根目录 20+ 个配置/说明文件、src/spark 扁平 30+ 模块、docs 中英文混杂、probe 类临时产物散落。建议如下（按见效速度排序）：

① 历史管理：git_memory 的任务级分支+squash 已是亮点，保持为默认路径；补 release-please 自动化版本与 CHANGELOG，tag 即 release，五平台脚本随 tag 推送；合并历史按 半年一次 squash 到 archive 分支，主历史保持可读。

② 仓库结构：src/spark 内按 core/tools/providers/memory/interfaces（web/tui/cli/desktop）四层分包，杜绝 interfaces 反向 import core 之外的层；根目录收敛为「pyproject + README×4 + CHANGELOG + Makefile + 两个 spec」，其余进 docs/ 与 .github/。

③ 会话数据：SQLite WAL 长期膨胀，给 store 加 vacuum 与 90 天归档策略，检查点快照目录加 LRU 清理（现在只进不出，磁盘会缓慢吃满）。

④ 文档：docs/ 按 en/ 与 zh/ 分离已有雏形，补 ADR 编号规范（现有 decisions/ 目录），README 的截图与功能宣称在每次 release 前走一遍核对清单。

八、生态建设路径

生态位选择：做 MCP 生态里的「本地执行层」+「中文模型最优路由」，而不是再造一个插件市场。分四步：

① 短期（1 个月）：Skills 兼容 Claude Code marketplace 格式（已部分支持），内置 10 个高质量中文场景技能（代码审查/测试生成/提交信息/文档翻译），让首次体验就有产出。

② 中期（3 个月）：把 hooks + skills 打磨成稳定插件 API 并冻结版本，写 3 个官方示范插件；开 GitHub Discussions 建技能分享区。

③ 渠道互补：mcp-serve 让 Codex/Claude Code 等竞品的用户也能把 Spark 当工具箱用，先把人吸进来再谈切换。

④ 长期：模型路由矩阵开放为社区可提交的 registry（类似 models.dev），国产模型厂商有动力自己维护条目——这是竞品不会做的差异化生态位。

九、疑问与建议（需要你拍板的事）

问题 1：是否接 Anthropic 原生协议？现在只有 OpenAI 兼容一路，Claude 系模型经 litellm 转换会丢失部分能力（交错思考、缓存控制）。若目标用户是国产模型人群，可以不接，省一条维护线。

问题 2：Web 多会话并发要不要做？现在单进程单 loop，如果目标场景是个人本机，建议明确「单会话为主」并文档化，避免用户误当服务端用。

问题 3：三个 GitHub 账号的分工是什么？若都用于推送，仓库归属混乱会稀释 star；建议主仓库归 badhope，另两个账号只做 fork 镜像或 CI 冗余。

问题 4：商业化要不要想？本地工具开源 + 企业版（审计日志/团队策略中心/SSO）是这类产品最顺的路，但早期别分心。

下一步建议（按依赖顺序）：修三个 P0（合计改动约 30 行）→ 补系统提示词 → PyPI 发布链路 → 真实 usage 回填 → 消息回溯 UI。这五件事做完，产品完成度会从「能用的 Demo」跨到「敢推荐给同事」。